

GRUPPO 31

PSS

Assignment 3 - Clean Code

Contents

1	Applicazione scelta	2
2	Classe JsonParser.java	3
2.1	Ruolo	3
2.2	Codice originale	3
2.3	Analisi di Clean Code	5
2.4	Code Refactoring	5

1 Applicazione scelta

2 Classe JsonParser.java

2.1 Ruolo

2.2 Codice originale

```
1 package it.marcosoft.ticketwave.NetworkActivity;
2
3 import android.content.Context;
4 import android.util.Log;
5
6 import androidx.recyclerview.widget.RecyclerView;
7
8 import com.android.volley.Request;
9 import com.android.volley.Response;
10 import com.android.volley.VolleyError;
11 import com.android.volley.toolbox.JsonObjectRequest;
12
13 import org.json.JSONArray;
14 import org.json.JSONObject;
15
16 import java.util.ArrayList;
17 import java.util.List;
18
19 import it.marcosoft.ticketwave.EventModel.Event;
20
21 /**
22  * Utility class for parsing JSON responses from the Ticketmaster API.
23  */
24 public class JsonParser {
25     // API key for accessing the Ticketmaster API
26     private static final String API_KEY = "apikey=KqtxCDlnofSteZ63m7gmezFR8PR34o78";
27
28     // Endpoint for the API request
29     private final String root;
30
31     // Query parameters for the API request
32     private final List<String> queryParams;
33
34     // List to store the parsed events
35     private final List<Event> eventsList;
36
37     // Android application context
38     private final OnEventsParsedListener onEventsParsedListener;
39
40     // RecyclerView to display the parsed events
41
42     /**
43      * Constructor for the JsonParser class.
44      *
45      * @param root The root endpoint for the API request.
46      * @param queryParams List of query parameters for the API request.
47      */
48     public JsonParser(String root, List<String> queryParams, OnEventsParsedListener
49 ↵ listener) {
50         this.root = root;
51         this.queryParams = queryParams;
52         this.eventsList = new ArrayList<>();
```

```

52     this.onEventsParsedListener = listener;
53 }
54
55 /**
56  * Creates a JsonObjectRequest for the API call.
57  *
58  * @return JsonObjectRequest for the API call.
59  */
60 public JsonObjectRequest jsonParse() {
61     // Build the URL for the API request
62     StringBuilder urlBuilder = new
63         ↳ StringBuilder("https://app.ticketmaster.com/").append(root).append("?");
64     for (String queryParam : queryParams) {
65         urlBuilder.append(queryParam).append("&");
66     }
67     String url = urlBuilder.append(API_KEY).toString();
68     Log.d("api", url);
69
70     // Create a JsonObjectRequest for the API call
71
72     return new JsonObjectRequest(
73         Request.Method.GET, url, null, new Response.Listener<JSONObject>() {
74
75             @Override
76             public void onResponse(JSONObject response) {
77                 try {
78                     // Parse the JSON response based on the API endpoint
79                     if ("discovery/v2/events".equals(root)) {
80                         JSONObject embedded = response.getJSONObject("_embedded");
81                         JSONArray events = embedded.getJSONArray("events");
82                         for (int i = 0; i < events.length(); i++) {
83                             JSONObject eventObj = events.getJSONObject(i);
84                             Event event = new Event(eventObj);
85                             eventsList.add(event);
86                         }
87                     } else {
88                         // Handle unknown API endpoint
89                         throw new Exception();
90                     }
91                 } catch (Exception e) {
92                     // Handle the exception (e.g., log it)
93                     Log.e("JsonParser", "Error parsing JSON", e);
94                 }
95
96                 if (onEventsParsedListener != null) {
97                     onEventsParsedListener.onEventsParsed(eventsList);
98                 }
99             }, new Response.ErrorListener() {
100
101                 @Override
102                 public void onErrorResponse(VolleyError error) {
103                     // Handle the Volley error (e.g., log it)
104                     Log.e("JsonParser", "Volley error", error);
105                 }
106             });
107
108     public interface OnEventsParsedListener {

```

```

109     void onEventsParsed(List<Event> events);
110 }
111 }

```

2.3 Analisi di Clean Code

La classe presenta diversi problemi di Clean Code, analizziamo quindi il codice presente al suo interno.

La funzione **jsonParse** risulta troppo massiccia e complessa, rendendo difficile capire a primo impatto cosa faccia e presentando diversi aspetti su cui poter agire.

Per iniziare il nome non riflette il compito che la funzione svolge, *jsonParse()* suggerisce che la funzione analizzi del codice JSON, possibilmente estraendo delle informazioni, ma al contrario la funzione esegue molte altre operazioni non inerenti all'analisi del JSON, un nome migliore potrebbe essere *getEventsFromAPI()*

Come anticipato, la funzione viola il **SRP (Single Responsibility Principle)**, si occupa infatti di:

- creare l'URL della API da chiamare;
- scrivere sul Log informazioni di debug;
- gestire la connessione HTTP verso la API;
- effettuare la chiamata alla API di TicketMaster;
- gestire la logica di parsing della risposta JSON;
- gestire la logica di eventuali errori di connessione o risposta;

Inoltre la funzione utilizza stringhe *embedded* per gestire la richiesta, creazione e parsing delle risposte, che rendono modifiche futura più laboriose e complesse; sarebbe opportuno usare variabili globali o costanti.

Elevato annidamento delle funzioni, dovuto alla gestione degli errori e parsing del JSON eseguita con lambda functions, che complicano esponenzialmente la leggibilità del codice, risolvibile estraendo i vari compiti in funzioni singole e separate.

La riga 84 presenta inoltre un grosso problema di integrità e sicurezza, gli eventi vengono infatti modificati direttamente sulla variabile originale, senza passare per una variabile locale intermedia, questò rischia di introdurre difficoltà di debug e problemi di concorrenza. La riga 26 presenta anch'essa un problema simile, il valore della API key non dovrebbe essere hardcoded in chiaro nella classe.

I nomi di variabili e funzioni sono perlopiù significativi e coerenti con il contesto di utilizzo, con alcune eccezioni quali: *Log.d* e *Log.e* che potrebbero risultare poco chiari.

Dal punto di vista dei commenti, la funzione presenta un commento generale che spiega lo scopo della funzione in maniera molto ambigua e generica, all'interno vi sono diversi commenti per delimitare e specificare le funzioni di ogni blocco di codice, ciò risulta ridondante e superfluo ma necessario data la complessità della funzione.

2.4 Code Refactoring